

1..what we observed now is variant 2 of sleeping Barber prblm ie. 2 barbers with shared tools which is the actual twist here..

Initially, both Barber 1 and Barber 2 are sleeping because there are no customers.

When Customer 1 arrives for Barber 1. The customer signals the semaphore, which wakes Barber 1.

Barber 1 acquires the shared tools and starts serving Customer 1.

After some time, Customer 2 arrives for Barber 2. The semaphore signals Barber 2, so Barber 2 wakes up and becomes ready to serve Customer 2.

However, the tools are currently being used by Barber 1. Therefore, Barber 2 cannot acquire the required resources and must wait.

2..

At this point, Barber 2 is **blocked by OS**, waiting for the resources to become available. This is called **process blocking due to resource unavailability**.

The blocking is actually useful bcz barber 2 does not continuously use the CPU while waiting..

When Barber 1 finishes serving Customer 1, he releases the tools.

The waiting Barber 2 can now be unblocked by some signals or os and acquire the tools. He can then serve Customer 2..

3..

Cn u guess the what concept is depicted here to audi...

yes, deadlock.. B1 is holding the comb and

waiting for the scissors. B2 is holding the scissors and waiting for the comb. This satisfies all four classic deadlock conditions:

Mutual Exclusion how?

only one barber can hold a tool at a time.

Hold and Wait how

each is holding one tool while waiting for the other.

No Preemption how

nobody can force the other to give up their tool.

Circular Wait how

B1 waits on B2, B2 waits on B1 — a closed loop.

So here is the indefinite wait...

The solution is:

One solution is resource ordering..

Here in our shop the rule is: first take the scissors, and only after taking the scissors, take the comb..

Both Barber 1 and Barber 2 must follow this same order..

//Now act Barber 1 return tool to semo  
And barber 2 to takes comb

//Now asking audience what if he again takes tool

We:

“What do you think this situation is called?”

Audience:

“Starvation!”

yes Starvation...so the final solution to avoid

deadlock and the Starvation caused by ordering of tools is avoided by instead of treating the two tools as two independent resources, we treat them as **one complete set of tools**

“The two tools are **binded together as a single resource.**”

“Therefore, a barber cannot take just one tool.”

“The barber requests the **entire tool set** from Semo.”

“If the complete set is available, Semo gives tool set to that barber.

#### 4..Conclusion

